

RAPPORT DE FIN DE MODULE

Module : Renforcement 2 | Algorithmique et Programmation

Réalisé par :

Hiba BEN DRIOUICH

Assuré par :

Pr. Sidi Mohamed SNINEH

Table des matières :

Introduction	3
Axe 1 : Fondements de l’algorithmique et de la programmation	4
1. Définition	4
2. Importance pédagogique	4
3. Passage de l’algorithme au programme	4
Axe 2 : Structures de données et programmation en Python	5
1. Les modules en Python	5
2. Les structures de données	5
3. Les listes	5
4. Tableaux et listes chaînées	6
5. Les tuples :	6
a. Création de tuples	6
b. Accès aux éléments	6
c. Parcours d'un tuple	6
d. Utilisations des tuples	6
e. Comparaison Listes vs Tuples	6
6. Les Dictionnaires	7
a. Création d'un dictionnaire	7
b. Ajout, modification et suppression	7
c. Accès aux éléments	7
d. Parcours d'un dictionnaire	7
e. Test d'appartenance et copie	7
7. Les Ensembles (Sets)	8
a. Création d'un ensemble	8
b. Ajout et suppression	8
c. Opérations mathématiques	8
d. Test d'appartenance	8
8. Tableau comparatif des structures de données Python	8
Axe 3 : Portée du module	10
1. Une vision fondatrice pour la formation	10
2. Portée pédagogique des apprentissages	10
3. Développement de compétences transversales	10
Conclusion	10

Introduction

« L'enseignement de l'algorithmique et de la programmation constitue un pilier fondamental dans la formation en informatique. Il ne s'agit pas uniquement d'apprendre à écrire du code, mais de développer une pensée logique, structurée et rigoureuse permettant de résoudre des problèmes complexes. »

Ce module vise à nous doter des bases nécessaires en algorithmique ainsi que des compétences en programmation, notamment à travers le langage Python. Il permet de passer d'une logique abstraite de résolution de problèmes à une implémentation concrète, tout en développant des capacités d'analyse, de structuration et d'optimisation.

Axe 1 : Fondements de l'algorithmique et de la programmation

1. Définition

L'algorithmique est l'étude des méthodes et des étapes nécessaires pour résoudre un problème de manière systématique. La programmation, quant à elle, consiste à traduire ces algorithmes en instructions compréhensibles par la machine à l'aide d'un langage de programmation.

2. Importance pédagogique

Ce module joue un rôle essentiel dans :

- Le développement de la pensée logique
- La structuration du raisonnement
- La capacité à décomposer un problème complexe
- L'autonomie dans la résolution de problèmes

3. Passage de l'algorithme au programme

L'apprentissage repose sur une progression allant de :

- L'analyse du problème
- La conception de l'algorithme
- L'implémentation en Python
- Le test et la validation

Axe 2 : Structures de données et programmation en Python

1. Les modules en Python

Les modules permettent d'organiser et de réutiliser le code, facilitant ainsi la structuration des programmes et leur maintenance.

Ils offrent une meilleure lisibilité et favorisent le travail modulaire.

Types de modules :

- ▶ Modules intégrés (built-in) : math, os, sys, random...
- ▶ Modules définis par l'utilisateur : fichiers .py créés par le développeur

Syntaxes d'importation :

```
import nom_module  
import nom_module as alias  
from nom_module import nom_var, nom_fct  
from nom_module import *
```

2. Les structures de données

Une structure de données est une manière spécifique d'organiser et de stocker des données dans un ordinateur pour qu'elles puissent être utilisées efficacement. Le choix d'une structure dépend de la nature des données et des opérations requises (recherche, insertion, suppression).

Catégorie	Types	Caractéristiques
Primitives	int, float, bool, str	Variables simples, valeurs uniques
Linéaires	Listes, Piles, Files	Séquentielles, accès selon l'ordre
Non linéaires	Arbres, Graphes	Relations hiérarchiques ou interconnectées
Associatives	Dictionnaires, Sets	Indexées par clés, non ordonnées

3. Les listes

Les listes sont des structures dynamiques permettant de stocker des collections ordonnées d'éléments de types variés. Elles offrent une grande flexibilité : ajout, suppression et accès aux éléments par indice. Elles constituent la base de nombreuses structures comme les piles et les files.

Opérations fondamentales sur les listes :

Opération	Fonction	Description
Créer	CréerListe(L)	Initialiser une nouvelle liste vide
Ajouter en tête	ajoutEntete(L, val)	Insérer un élément au début
Supprimer	suppelt(L, val)	Retirer un élément de la liste
Tester si vide	ListeVide(L)	Vérifie si la liste est vide
Compter	TailleListe(L)	Retourne le nombre d'éléments

4. Tableaux et listes chaînées

- **Tableaux** : accès rapide, taille fixe
- **Listes chaînées** : structure dynamique, accès séquentiel

Ce choix impacte directement les performances des algorithmes.

5. Les tuples :

Un tuple est un type de données composite, semblable à une liste, mais immuable : ses éléments ne peuvent pas être modifiés après création. Les tuples sont délimités par des parenthèses.

a. Création de tuples

```
t1 = (1, 3, 5, 7, 9)
t2 = (10, 20, 'A', 'B', True, 5.23)
t3 = tuple([1, 2, 3]) # depuis une liste
```

b. Accès aux éléments

```
print(t1[0]) # Premier élément : 1 print(t1[-1]) # Dernier élément : 9 print(t1[1:3]) # Slicing : (3, 5)
```

c. Parcours d'un tuple

```
for element in t1: print(element) # Avec range et len for i in range(len(t1)): print(t1[i])
```

d. Utilisations des tuples

- ▶ Affectation parallèle : a, b = 1, 2
- ▶ Retourner plusieurs valeurs depuis une fonction
- ▶ Utilisation comme clés de dictionnaire (car immuables)

e. Comparaison Listes vs Tuples

Critère	Listes	Tuples
Mutabilité	Modifiable	Immuable
Syntaxe	[]	()
Mémoire	Plus d'espace	Plus léger
Vitesse	Plus lente	Plus rapide
Usage	Données variables	Données fixes
Clé dict	Non	Oui

6. Les Dictionnaires

Les dictionnaires constituent un type de données composite contenant des éléments non-ordonnés indexés par des clés. Contrairement aux listes indexées par des entiers, les dictionnaires utilisent des clés de tout type immuable (chaîne, entier, tuple). Ils sont modifiables.

a. Création d'un dictionnaire

```
d1 = {"id": 10, "nom": "Ali"}
```

b. Ajout, modification et suppression

```
# Ajout / modification : d2["Couleur"] = "Noir"
# Suppression del d2["Couleur"] d2.pop("Modele")
# Vider le dictionnaire d2.clear()
```

c. Accès aux éléments

```
# Accès par clé : print(d1["nom"])
print(d1.get("age", 0))
```

d. Parcours d'un dictionnaire

```
# Parcours des clés : for cle in d1:
    print(cle, ":", d1[cle])
# Parcours des paires clé/valeur : for cle, val in d1.items():
    print(cle, "-->", val)
```

e. Test d'appartenance et copie

```
# Test d'appartenance : if "nom" in d1: print("La clé 'nom' existe")
# Copie d_copie = d1.copy()
```

7. Les Ensembles (Sets)

Les objets de type ensemble (set) modélisent la notion mathématique d'ensemble. Les éléments ne sont pas ordonnés et ne peuvent pas être accédés par un indice : on peut seulement vérifier si une valeur appartient ou non à l'ensemble. Les doublons sont automatiquement éliminés.

a. Création d'un ensemble

```
s1 = {1, 2, 3, 4, 5}
s2 = set([1, 2, 2, 3, 3])
s3 = set()
```

b. Ajout et suppression

```
# Ajout d'un élément : s1.add(6)
# Suppression :
# (erreur si absent) s1.remove(6)
# (pas d'erreur) s1.discard(99)
# Retirer un élément aléatoire s1.pop()
```

c. Opérations mathématiques

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
# Union print : (A | B)
# Intersection : print(A & B)
# Différence : print(A - B)
# Différence symétrique : print(A ^ B)
```

d. Test d'appartenance

```
if 3 in A: print("3 est dans A")
```

8. Tableau comparatif des structures de données Python

Critère	Listes	Tuples	Dictionnaires	Ensembles (Sets)
Syntaxe	[]	()	{ clé: val }	{ val } / set()
Ordonné	Oui	Oui	Non (Python < 3.7)	Non
Modifiable	Oui	Non	Oui	Oui
Doublons	Autorisés	Autorisés	Clés uniques	Interdit
Accès	Par indice	Par indice	Par clé	Non indexé

Critère	Listes	Tuples	Dictionnaires	Ensembles (Sets)
Clé de dict	Non	Oui	Non	Non
Usage typique	Données ordonnées	Données fixes	Paires clé/valeur	Éléments uniques

Axe 3 : Portée du module

1. Une vision fondatrice pour la formation

Le module d'algorithmique et de programmation constitue un socle fondamental pour toute formation en informatique. Il conditionne la compréhension des domaines avancés tels que le développement logiciel, les bases de données, l'intelligence artificielle et les systèmes informatiques.

2. Portée pédagogique des apprentissages

Ce module se distingue par sa forte portée pédagogique, en développant chez l'apprenant une démarche structurée de résolution de problèmes. L'accent est mis sur la compréhension des concepts plutôt que sur la simple écriture du code, favorisant ainsi un apprentissage durable.

3. Développement de compétences transversales

Au-delà des compétences techniques, ce module permet de développer :

- La rigueur et la logique
- L'autonomie
- La capacité d'analyse
- La résolution de problèmes

La programmation devient ainsi un outil de réflexion et non seulement un moyen technique.

Conclusion

Le module d'Algorithmique et Programmation constitue une étape essentielle dans la formation en informatique. Il permet d'acquérir des bases solides tant sur le plan conceptuel que pratique.

L'étude approfondie des structures de données Python — listes, piles, files, tuples, dictionnaires et ensembles — offre une compréhension globale des outils mis à la disposition du programmeur. Chaque structure répond à des besoins spécifiques :

- Les listes et tuples pour les séquences ordonnées
- Les dictionnaires pour les associations clé-valeur
- Les ensembles pour gérer des collections sans doublons avec des opérations mathématiques

En développant une pensée algorithmique rigoureuse et des compétences en programmation, ce module prépare efficacement à l'apprentissage de technologies plus avancées et à une pratique professionnelle structurée.